

Prompt Engineering Cheat Sheet

16 tables · 108 concepts · Updated 2026-05-31 · [cheatgrid.com](#)

Prompt engineering is the practice of designing and optimizing textual instructions that guide large language models (LLMs) and other AI systems to generate desired outputs. Born from the rise of transformer-based models like GPT, Claude, and Gemini, prompt engineering has evolved from simple question-answer patterns into a sophisticated discipline involving reasoning frameworks, output control, and security considerations. As models grow more capable, the field is converging with **context engineering** — the broader practice of shaping all information a model receives — making the structure, format, and context of prompts as important as the words themselves.

CONTENTS

Core Prompting Approaches	1
Reasoning and Decomposition Techniques	2
Output Control and Formatting	3
Advanced Reasoning Patterns	4
Message Roles and Context Structure	5
Prompt Chaining and Workflow Orchestration	5
Sample Selection and Example Design	6
Generation Parameters and Sampling	6
Multimodal and Vision-Language Prompting	7
Safety and Robustness	7
Emotion and Persona Techniques	8
Optimization and Automation	8
Specialized Patterns and Emerging Techniques	9
Prompting for Reasoning Models	9
Domain-Specific Applications	10
Anti-Patterns and Common Pitfalls	10
References	11

TABLE 1

Core Prompting Approaches

Foundational ways to shape an LLM's answer using the wording of a single prompt, before reaching for reasoning chains or tools. They differ mainly in how much you show the model (no examples, one, or several), how you frame the request (persona, situational context, explicit rules), and whether the model is asked to clarify the question for itself first.

Technique	Example	Description
Zero-shot prompting	Translate to French: Hello	- Model performs task without examples, relying solely on pre-training knowledge - fast but less reliable for complex or domain-specific tasks
Few-shot prompting	English: cat → French: chat English: dog → French: chien English: bird → ?	- Provides 2–5 example input-output pairs before the query - significantly improves accuracy and consistency for nuanced tasks
One-shot prompting	Example: "angry" → negative Classify: "delightful" → ?	Single demonstration example - useful when task is straightforward but model needs format guidance

Technique	Example	Description
Role prompting	You are an expert oncologist. Explain CAR-T therapy.	- Assigns a persona or expertise to the model - most effective for controlling tone, style, and output format rather than expanding factual knowledge
Instruction following	List three benefits. Use bullet points. Keep under 50 words.	- Explicit directives on what, how, and constraints - essential for controlling output length, format, and style
Contextual prompting	Background: User is a beginner. Task: Explain neural networks.	Provides situational information (audience, constraints, domain) to shape response appropriately
Rephrase and Respond (RaR)	Rephrase and expand this question, then answer: Why is the sky blue?	- Model rephrases the question before answering - improves accuracy by resolving ambiguity in the original phrasing

TABLE 2

Reasoning and Decomposition Techniques

These prompts get a model to work through a problem instead of guessing an answer in one leap. Some show step-by-step reasoning, others split a task into smaller parts, branch and backtrack across options, or interleave reasoning with real tool calls. Their gains depend heavily on model scale, and a written reasoning trace is not a guaranteed account of what the model actually did.

Method	Example	Description
Chain-of-Thought (CoT)	Q: $23 + 47 = ?$ A: $23 + 47 = 20 + 40 + 3 + 7 = 60 + 10 = 70$	- Prompts model to show step-by-step reasoning - an emergent ability that helps large models on math and logic, but can fail or hurt on small models
Zero-shot CoT	Let's think step by step.	Triggers reasoning without examples - effective shortcut when few-shot is impractical; redundant on reasoning models (o1/o3/R1)
Self-consistency	Generate 5 answers via CoT → select majority answer	- Samples multiple independent reasoning paths and takes the majority answer - improves reliability, but costs N times the tokens and latency
Tree of Thoughts (ToT)	Evaluate 3 approaches → explore best 2 → backtrack if stuck	- Models reasoning as branching exploration with self-evaluation and backtracking - uses search over partial paths, handling planning and multi-path problems
Least-to-Most prompting	Step 1: Simplify equation Step 2: Solve for x using Step 1	- Decomposes a problem into ordered subproblems, each fed the previous answer - generalizes to problems harder than the examples shown
ReAct (Reasoning + Acting)	Thought: Need population data Action: search("France population") Observation: 67M → Answer	- Interleaves reasoning traces with tool actions and observations - grounds reasoning in retrieved results, reducing hallucination
Plan-and-Solve (PS+)	First, devise a plan to solve this. Then carry out the plan step by step.	- Model plans subtasks before executing them - a zero-shot method that reduces the missing-step and calculation errors of zero-shot CoT
Step-back prompting	Before answering, what general principles apply to this problem?	- Model identifies high-level concepts or first principles before specifics - improves reasoning on knowledge-intensive and abstract problems
Graph of Thoughts (GoT)	thought_1 + thought_2 → aggregated_insight Loop back for refinement	- Organizes reasoning as a directed graph that can merge branches and loop back - most flexible for complex interdependent reasoning

Method	Example	Description
Thread of Thought (ThoT)	Walk me through this context step by step, summarizing as you go.	- Segments and analyzes long or chaotic contexts methodically - plug-and-play technique for tasks with extended or noisy input
Auto-CoT	Cluster questions by diversity → auto-generate CoT demos	Automatically constructs chain-of-thought demonstrations without manual effort - samples diverse questions so the occasional wrong auto-generated chain does little harm
Self-Ask	Are follow-up questions needed? Yes: What is...? → intermediate answer Final answer: ...	- Model generates and answers sub-questions before the main answer - improves compositional and multi-hop reasoning

TABLE 3

Output Control and Formatting

These techniques shape what the model returns and how it is structured, so downstream code can parse it and humans can read it. A key distinction runs through the table: prompt-only instructions (asking for JSON, a length, or "do not" rules) are soft requests the model can miss, while API-level features like structured outputs and max_tokens enforce hard constraints.

Technique	Example	Description
Structured output (JSON)	Return as JSON: {"name": str, "age": int}	- Enforces a specific schema (JSON, XML, YAML) - only schema-enforced structured outputs (constrained decoding) guarantee conformance; a prompt-only "Return JSON" can still emit invalid or extra text
XML tag structuring	<context>text</context> <instructions>summarize</instructions>	- Wraps prompt sections in semantic XML tags - reduces ambiguity by marking boundaries; an Anthropic best practice especially effective with Claude, but clear delimiters help most models
Delimiters and sections	### Input text ### Output summary	- Uses markers (###, ` , ---) to separate sections - reduces ambiguity about what content the model should process vs. generate
Output length control	Summarize in exactly 3 sentences. Keep under 100 tokens.	- Specifies word/sentence/token count - a stated count is a soft target the model may miss; max_tokens is a hard truncation that can cut output mid-word and break JSON
Format templates	<summary> <title>...</title> <body>...</body> </summary>	- Provides a markup skeleton for the model to fill - keeps nested or hierarchical output consistent; especially effective with XML
Enumerated instructions	1. Extract entities 2. Classify sentiment 3. Return as table	- Numbered steps clarify sequence and expectations - improves task adherence when multiple operations are required
Negative prompting (constraints)	Do NOT include personal opinions. Avoid bullet points.	- Specifies what to exclude from output - unreliable on its own because models handle negation poorly; pair with positive framing (say what to include)

TABLE 4

Advanced Reasoning Patterns

These patterns push a model past a single answer by adding structure: writing its own prompts or examples, critiquing and verifying its work, offloading hard computation to code, or trading layout for speed. Knowing what each one actually changes (and where it quietly fails) is what separates a reliable pipeline from a fragile one.

Pattern	Example	Description
Meta-prompting	Generate a prompt to classify movie reviews.	- Model writes or optimizes prompts for a task - enables iterative self-improvement and automated prompt engineering
Generated knowledge prompting	First, list relevant facts about photosynthesis. Now answer: What role does chlorophyll play?	- Model generates intermediate knowledge before answering - improves factual accuracy on knowledge-intensive queries, using its own training (not external retrieval)
Self-Refine	Draft → Critique your draft → Revise based on feedback → repeat	- Model iteratively generates, critiques, and refines its own output - no external model needed; lifts quality on generation tasks, but self-critique alone does not reliably fix reasoning errors
Chain of Verification (CoVe)	Answer → generate verification questions → answer each independently → revise	- Model plans verification questions, answers them independently of the draft, then revises - significantly reduces hallucinations in factual tasks
Directional stimulus prompting	Keywords: protein, folding, disease Write an abstract.	- Provides hints or cues (keywords, themes) to steer generation toward desired content - a small trained policy model can generate the hints for a frozen LLM
Program-Aided Language (PAL)	Write Python to solve: "If $x^2 = 16$, find x " def solve(): return sqrt(16)	- Model generates executable code as the reasoning step - offloads arithmetic to an interpreter that runs the code, for higher accuracy
Skeleton-of-Thought (SoT)	First: generate outline with 5 sections. Then: write each section in parallel.	- Creates structural outline first, then parallelizes content generation - reduces latency by up to 2.4x for long outputs
Chain of Density (CoD)	Summary 1: sparse (50 words) Summary 2: denser (same length, +3 entities) Iterate 5 times	Iteratively packs more entities into a fixed-length summary - produces human-preferred summaries by the later steps
Active-Prompt	Measure uncertainty on unlabeled questions → annotate most uncertain → add to few-shot pool	- Uses uncertainty sampling to select which examples a human should annotate - improves few-shot performance with minimal human labeling
Analogical prompting	Recall relevant problems similar to this, then solve by analogy.	- Model self-generates relevant examples before solving the task - eliminates manual few-shot curation; improves math and code reasoning
Cumulative reasoning	Generate propositions iteratively → verify each → accumulate into final answer	- Uses a proposer, verifier, and reporter to build the answer from verified steps - verifying each proposition before accumulating it is what sets it apart from plain chain-of-thought

TABLE 5

Message Roles and Context Structure

Chat models read a list of role-tagged messages rather than one block of text. The `system` (and newer `developer`) role sets standing behavior, `user` carries the live request, and `assistant` holds the model's prior replies. Because each request is stateless, your app resends the whole list every turn to maintain context, and higher-privilege roles outrank the user role when instructions conflict.

Role	Example	Description
System message	You are a helpful assistant specializing in Python.	• Sets global behavior, persona, and constraints • applied before all user messages as persistent context, but it is guidance, not a security boundary
User message	How do I reverse a list in Python?	• Contains user query or command • the primary input the assistant responds to
Assistant message	Use <code>list.reverse()</code> or slicing: <code>lst[::-1]</code>	• Model's previous response, supplied back as history • you can also write one to prefill or steer the next answer
Multi-turn context	[user] "Define recursion" [assistant] "..." [user] "Give example"	• Each request is stateless, so the client resends the full history every turn • longer chats cost more tokens and can exceed the context window
Developer message	[developer] "Always respond in JSON format"	• OpenAI's newer app-developer instruction role • ranks above user messages in the instruction hierarchy and is meant to win conflicts

TABLE 6

Prompt Chaining and Workflow Orchestration

Once a task is too big for one prompt, you compose several model calls into a pipeline. These patterns range from simple sequential chains to retrieval, tool use, routing, and self-directing agents. A key theme: the model proposes structured steps, but your code executes tools, routes branches, and enforces stop conditions.

Technique	Example	Description
Prompt chaining	Prompt 1: Extract entities → output_1 Prompt 2: Classify entities from {output_1}	• Decomposes a task into sequential LLM calls • each prompt's output feeds the next, so steps stay simple and easy to debug
Retrieval-Augmented Generation (RAG)	1. Retrieve docs about "mitochondria" 2. Prompt: "Using {docs}, explain ATP synthesis"	• Fetches external documents at query time and adds them to the prompt • grounds answers in current or proprietary data without retraining the model
Function calling (tool use)	tools: [{"name": "get_weather", "parameters": {"location": "string"}}]	• Model selects a structured tool schema and emits name plus arguments • your application code runs the tool, so validate arguments before executing
Agentic workflows	Agent: Plan → Act → Observe → Refine → Act	• Model directs its own steps, choosing tools based on each result • loops toward a goal, so a max-iteration cap is needed to avoid runaway cost
Conditional branching (routing)	If sentiment=negative: call escalation_prompt Else: call thank_you_prompt	• Classifies the input, then routes it to a specialized prompt • separates concerns so each branch stays focused on one kind of case
ReWOO (Reasoning Without Observation)	Plan all tool calls upfront → execute → synthesize	• Decouples planning from observation • a planner writes the full plan with placeholders, workers run tools, a solver combines results, cutting LLM calls vs ReAct

TABLE 7

Sample Selection and Example Design

Which examples you put in a few-shot prompt, and in what order, often moves accuracy more than how many you add. This table covers the main ways to choose demonstrations, from query-matched and balanced sets to contrastive pairs, plus the biases that make order and label balance matter.

Strategy	Example	Description
Similarity-based selection	Choose examples most similar to query via embedding distance	- Provides contextually relevant demonstrations - often outperforms random, but similar examples cluster and can lose diversity
Stratified sampling	2 positive, 2 negative, 1 neutral sentiment	- Ensures balanced coverage of categories - counters majority-label bias when data is imbalanced
Contrastive examples	Correct: "Step A → B → C" Incorrect: "Step A → C (missing B)"	- Shows both correct and incorrect cases - helps the model see which reasoning steps to avoid
Example ordering	Place most relevant or recent examples last	- LLMs exhibit recency bias - reordering the same examples can swing accuracy from near chance to near best
Random sampling	Pick 5 random examples from dataset	Baseline approach - fast but mirrors data skew and ignores query relevance

TABLE 8

Generation Parameters and Sampling

These settings control how a model turns its next-token probabilities into actual text: how much randomness to allow, which low-probability tokens to discard, how long to keep going, and when to stop. Tuning them well is the difference between focused, parseable output and creative-but-unreliable rambling.

Parameter	Example	Description
Temperature	temperature=0.0 (deterministic) temperature=1.0 (creative)	- Controls randomness, not answer quality - lower = more focused/repetitive, higher = more diverse/creative - typical range 0–2; even 0 is not guaranteed bit-for-bit identical across runs
Top-p (nucleus sampling)	top_p=0.9	- Keeps the smallest token set whose cumulative probability $\geq p$, then samples from it - adapts the candidate count to the model's confidence - vendors recommend tuning temperature or top_p, not both
Max tokens	max_tokens=150	Hard cap on output length that truncates the moment it is hit - not a target length; can cut mid-sentence and break JSON - prevents runaway generation and controls cost
Top-k sampling	top_k=40	- Restricts sampling to the k most likely tokens (a fixed count) - simpler than top-p but a blunt cutoff that ignores the distribution's shape
Frequency penalty	frequency_penalty=0.5	Reduces repetition by penalizing tokens in proportion to how often they have already appeared (count-based)
Presence penalty	presence_penalty=0.6	Encourages topic diversity with a flat one-time penalty applied once a token has appeared at all, regardless of count
Min-p sampling	min_p=0.05	- Keeps tokens above a fraction of the top token's probability (base value $\times$ top probability) - adaptive: strict when one token dominates, relaxed when the model is uncertain - pairs well with temperature > 1
Stop sequences	stop=["###", "\n\n"]	Terminates generation when a specified string is produced (content-based, unlike the length-based max-tokens cap) - useful for structured outputs and preventing runaway text

TABLE 9

Multimodal and Vision-Language Prompting

Multimodal prompting feeds a model more than text. You pass an image or audio clip alongside your question, and the model reasons over both. Keep in mind these models do not "see" or "hear" perfectly: they give approximate object counts, struggle with precise spatial detail, and can hallucinate text when reading documents, so verify anything high-stakes.

Approach	Example	Description
Image + text prompting	[image of chart] What trend does this show?	- Combines visual and textual input - model analyzes image content to answer text query
Visual question answering	[image of room] How many chairs are visible?	- Model performs object counting, detection, or scene understanding from image - Counts are approximate, so verify them
OCR and document understanding	[scanned receipt] Extract total amount.	- Reads and interprets text within images, including tables, forms, and structured documents - May hallucinate plausible but wrong values
Image captioning	[photo] Generate detailed caption.	Model produces natural language description of image content
Visual reasoning	[two images] Which object is larger?	- Requires comparison or relational reasoning across visual inputs - Precise spatial localization is unreliable
Audio prompting	[audio clip] Transcribe and summarize this meeting.	- Processes speech or audio input natively - supported by multimodal models like GPT-4o for transcription, analysis, and translation

TABLE 10

Safety and Robustness

Securing an LLM application means assuming its prompts and the data it reads are adversarial. These techniques cover the layered defenses that matter most: keeping untrusted input from overriding your instructions, validating what the model emits, training models to refuse harmful requests, and testing your system the way an attacker would before it ships.

Technique	Example	Description
Prompt injection defense	Use input handling , instruction delimiters, and privilege limits	- Mitigates attacks where input tries to override developer instructions or exfiltrate data - OWASP ranks prompt injection as LLM01, the top LLM risk
Output validation	Check output against a schema, encode it, or screen with a secondary LLM	Treats model output as untrusted before it reaches a browser, database, or shell, preventing XSS, SQL injection, or command execution
Constitutional AI principles	Model self-critiques against rules like Refuse harmful requests. Be helpful and honest.	A training method (RLAIF): the model critiques and revises its own answers against a set of principles, not a runtime word filter
Red-teaming prompts	Run adversarial probes such as injection and jailbreak attempts before launch	Adversarial testing to find vulnerabilities before attackers do . Expected by the NIST AI RMF and OWASP LLM Top 10
Jailbreak resistance	Detect attempts to bypass safety via role-play, encoding, or indirection	Models trained to recognize and refuse disguised harmful requests , targeting the model's safety rules (distinct from injection)
Indirect prompt injection defense	Separate trusted instructions from untrusted external data using privilege boundaries	- Prevents attackers from embedding hidden instructions in documents, emails, or tool outputs the model processes - The key risk for agentic and RAG systems

TABLE 11

Emotion and Persona Techniques

These techniques shape how a model speaks and reasons by giving it a role, an audience, or an emotional frame. They mostly steer tone, depth, and perspective, and their effect on factual accuracy is far weaker and less reliable than popular advice suggests.

Technique	Example	Description
Expert persona	You are a Pulitzer Prize-winning journalist. Write a headline.	• Assigns specific expertise or identity • mainly shapes tone, depth, and style, and does not reliably boost factual accuracy
Multi-persona prompting	Summon three experts (security, UX, backend). Have them collaborate on a review.	• One model simulates multiple expert personas collaborating in a single self-collaboration • produces more thorough, multi-perspective outputs
Emotional prompting	This is very important to my career. Please give your best answer.	• Adds emotional stakes or urgency • reported gains in earlier studies, but effects are mixed and model-dependent, often weaker on frontier models
Simulated Theory of Mind (SimToM)	Put yourself in the reader's shoes. What would they find confusing?	• Two-stage perspective-taking: filter context to what a character knows, then answer from that view • improves reasoning about beliefs and supports more empathetic responses

TABLE 12

Optimization and Automation

These methods move prompt work from hand-tuning to measured, repeatable engineering: tools that auto-generate and score prompts (APE, DSPy), ways to compare and version prompts in production (A/B testing, prompt versioning), a parameter-efficient training alternative (soft prompts), and an inference trick that reuses a repeated prefix to cut cost and latency (prompt caching).

Method	Example	Description
Automatic Prompt Engineering (APE)	Generate prompt candidates → score on a dataset → select best performer	• A model proposes instruction candidates, which are then scored and filtered on a validation set • replaces manual trial-and-error
DSPy framework	Define signatures → framework compiles and optimizes prompts from examples and a metric	• Declarative approach where prompts are compiled, then iteratively improved against a metric, not hand-written • discards variations that do not score better
A/B testing prompts	Run variant A vs B on the same inputs → measure accuracy, latency, cost → deploy winner	• Empirical comparison to select the best prompt for production • needs enough samples for statistical confidence, since LLM outputs vary
Prompt tuning (soft prompts)	Learn a few continuous embedding vectors prepended to the input while the model stays frozen	• Trains small learnable vectors, not readable text, leaving model weights frozen • parameter-efficient alternative to full fine-tuning
Prompt versioning	Track each prompt change as an immutable, identified version with eval metrics	• Manages prompt iterations in production • enables exact rollback, A/B testing, and regression tracking
Prompt caching	Place static system instructions first → variable content last	• Providers reuse the computed prefix for an exact match, cutting cost up to ~90% and latency up to ~80% • caches the input prefix, never the response; supported by OpenAI, Anthropic, Google

TABLE 13

Specialized Patterns and Emerging Techniques

These are newer or niche prompting patterns, several from single recent papers, that squeeze more reliability out of a model without touching its weights. They lean on tricks like self-reflection, voting across reasoning chains, picking complex examples, and even repeating the prompt, so treat the emerging ones as promising rather than settled and verify before production use.

Pattern	Example	Description
Reflexion	Review your answer. What could be improved? Revise → iterate	- Model self-critiques and writes a verbal reflection it stores in memory as context for the next attempt - reinforces the agent without any weight update or fine-tuning
Complexity-based prompting	Select few-shot examples with the most reasoning steps	- Prefers demonstrations with higher reasoning complexity (longer chains) - can also vote over the most complex chains at decoding; raises multi-step accuracy
Maieutic prompting	Generate explanation tree → prune contradictory branches	- Builds an abductive, recursive tree of explanations - frames the answer as a satisfiability problem over their logical relations to find the most consistent one
Universal Self-Consistency	Apply self-consistency to non-reasoning tasks (e.g., classification, extraction)	- Has the LLM itself pick the most consistent of several candidate answers - extends majority-voting benefits to free-form tasks where answers cannot be counted
Prompt repetition	What are the causes of inflation? What are the causes of inflation?	- Repeating the prompt twice gives a bidirectional-context effect in causal models, reported to help non-reasoning LLMs - doubles input token cost but adds no generated tokens or latency
DR-CoT (Dynamic Recursive CoT)	Recurse on sub-steps → truncate context → vote across reasoning chains	- Combines recursive refinement, dynamic context truncation within a token budget, and majority voting - helps parameter-efficient models rival larger ones; voting can still fail under shared model bias

TABLE 14

Prompting for Reasoning Models

Reasoning models such as OpenAI o1/o3, Claude with extended thinking, and DeepSeek-R1 think before they answer, so they reward concise goal statements over heavy step-by-step scaffolding. These techniques cover how to steer their hidden reasoning, tune its depth against cost and latency, and avoid instructions that older models needed but these models do not.

Technique	Example	Description
Goal-oriented prompting	Solve for x where $3x + 7 = 22$. Show the solution process and final result.	- State desired outcome clearly without prescribing steps - reasoning models (o1/o3/R1) perform best with concise goal statements
Extended thinking (budget tokens)	thinking: {type: "enabled", budget_tokens: 10000}	- Allocates a reasoning scratchpad for Claude models, drawn from <code>max_tokens</code> and at least 1024 - model thinks step-by-step in a hidden block before producing the answer
Reasoning effort control	reasoning_effort: "high"	- Adjusts how deeply the same model reasons before answering "low" for simple tasks, "high" for complex problems; controls cost and latency
Avoid explicit CoT instructions	Do not add "think step by step" to o1/o3/R1	- Reasoning models already reason internally - explicit CoT is redundant and can increase latency without benefit

TABLE 15

Domain-Specific Applications

Prompt engineering plays out differently across common LLM tasks. Code and data work reward low temperature and strict structure, summarization and translation lean on dedicated techniques like Chain of Density and few-shot terminology, and grounded question answering depends on retrieval. Knowing the right tool and setting per task is what separates a reliable pipeline from a flaky one.

Domain	Example	Description
Code generation	Write a Python function to merge two sorted lists.	• Produces runnable code, but output can look correct yet miss details like an import • favor low temperature and always test the result
Data extraction	Extract: name, email, phone from: "Contact John at john@ex.com"	• Pulls structured fields from unstructured text • structured outputs enforce a JSON schema at decode time, far more reliable than free-text or plain JSON mode
Summarization	Summarize this article in 2 sentences.	• Condenses long text into key points • Chain of Density packs more entities into a fixed length and reduces lead bias
Creative writing	Write a haiku about autumn.	• Generates poetry, stories, or dialogue • higher temperature adds variety, but too high turns coherent text into nonsense
Translation	Translate to German: "Good morning"	• Converts text between languages • few-shot examples of approved terminology improve accuracy on domain terms
Question answering	Based on: {document}, answer: Who founded the company?	• Provides a factual answer from context • RAG grounds answers in private or fresh sources, but is unneeded for general knowledge
Sentiment analysis	Classify sentiment: "I loved this movie!" → positive	• Determines emotional tone • few-shot with diverse examples improves handling of sarcasm and mixed reviews

TABLE 16

Anti-Patterns and Common Pitfalls

These are the prompt habits that quietly wreck output quality, run up cost, or produce confidently wrong answers. For each one, the fix is usually the opposite move: be specific, decompose, show examples, set limits, mark boundaries, ground recent facts with retrieval, and match sampling to the task. The last row is an evolving guideline, heavy step-by-step scaffolding helps weaker models but can over-constrain frontier ones.

Pattern	Example	Description
Vague instructions	Tell me about AI.	• Lacks specificity • produces generic, unfocused output; always specify scope, audience, or format
Overloading single prompt	Mixing 10 unrelated tasks in one prompt	• Splits the model's attention, so every task gets shallow output • better to chain or decompose into separate prompts
No examples for complex tasks	Zero-shot on nuanced classification	• Underperforms without demonstrations • 2-5 few-shot examples teach your exact criteria; more than that tends to plateau
Ignoring output length	No length constraint, leading to a 5000-word response	• Generates unnecessarily long outputs and runs up cost at scale • state the length in words; max-tokens only truncates, it does not shape length
Ambiguous delimiters	Input: text here Output: more text (no clear boundary)	• Model confuses what to process vs. generate • use ### or ` to separate sections

Pattern	Example	Description
Assuming knowledge cutoff awareness	What happened last week? (model trained months ago)	- Model cannot access real-time data and will confidently invent recent facts - ground it with RAG or tool use
Wrong parameters for task	Deterministic task with temperature=1.5	Excessive randomness where consistency is needed (temperature is not truthfulness) - tune temperature and top-p to the task
Excessive scaffolding for capable models	10-step procedural instructions for a frontier model on an open-ended task	- Over-constraining can hinder autonomous reasoning in frontier models - describe the desired result and let it choose the route; strict steps still suit procedural, schema-bound tasks

References

Official frameworks & canonical authors

- Output validation - <https://genai.owasp.org/llmrisk/llm052025-improper-output-handling/>
- Prompt injection defense - <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

Vendor documentation & tools

- Agentic workflows - <https://www.anthropic.com/research/building-effective-agents>
- Audio prompting - <https://platform.openai.com/docs/guides/audio>
- Constitutional AI principles - <https://www.anthropic.com/research/constitutional-ai-harmlessness-from-ai-feedback>
- Extended thinking (budget tokens) - <https://platform.claude.com/docs/en/build-with-claude/extended-thinking>
- Goal-oriented prompting - <https://developers.openai.com/api/docs/guides/prompt-engineering>
- Indirect prompt injection defense - <https://www.anthropic.com/research/prompt-injection-defenses>
- Min-p sampling - <https://huggingface.co/posts/joaogante/319451541682734>
- Prompt caching - <https://platform.openai.com/docs/guides/prompt-caching>
- Structured output (JSON) - <https://developers.openai.com/api/docs/guides/structured-outputs>
- Top-k sampling - https://huggingface.co/docs/transformers/generation_strategies
- Vague instructions - <https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api>
- XML tag structuring - <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>
- Assistant message - <https://developers.openai.com/api/docs/guides/conversation-state>
- Avoid explicit CoT instructions - <https://developers.openai.com/api/docs/guides/reasoning-best-practices>
- DSPy framework - <https://dspy.ai/>
- Function calling (tool use) - <https://developers.openai.com/api/docs/guides/function-calling>
- Image + text prompting - <https://huggingface.co/blog/vlms>
- Instruction following - <https://developers.openai.com/api/docs/guides/prompt-engineering/>
- Output length control - <https://developers.openai.com/api/docs/guides/reasoning>
- Prompt tuning (soft prompts) - https://huggingface.co/docs/peft/conceptual_guides/prompting
- System message - <https://learn.microsoft.com/en-us/azure/ai-factory/openai/concepts/advanced-prompt-engineering>
- User message - <https://developers.openai.com/api/docs/guides/text-generation>
- Visual question answering - <https://platform.openai.com/docs/guides/vision>

Academic papers & preprints

- Active-Prompt - <https://arxiv.org/abs/2302.12246>
- Automatic Prompt Engineering (APE) - <https://arxiv.org/abs/2211.01910>
- Chain of Verification (CoVe) - <https://arxiv.org/abs/2309.11495>
- Complexity-based prompting - <https://arxiv.org/abs/2210.00720>
- Example ordering - <https://arxiv.org/abs/2102.09690>
- Graph of Thoughts (GoT) - <https://arxiv.org/abs/2308.09687>
- Maieutic prompting - <https://arxiv.org/abs/2205.11822>
- Plan-and-Solve (PS+) - <https://arxiv.org/abs/2305.04091>
- ReAct (Reasoning + Acting) - <https://arxiv.org/abs/2210.03629>
- Rephrase and Respond (RaR) - <https://arxiv.org/abs/2311.04205>
- Self-Ask - <https://arxiv.org/abs/2210.03350>
- Self-Refine - <https://arxiv.org/abs/2303.17651>
- Simulated Theory of Mind (SimToM) - <https://arxiv.org/abs/2311.10227>
- Auto-CoT - <https://arxiv.org/abs/2210.03493>
- Chain of Density (CoD) - <https://arxiv.org/abs/2309.04269>
- Chain-of-Thought (CoT) - <https://arxiv.org/abs/2201.11903>
- DR-CoT (Dynamic Recursive CoT) - <https://www.nature.com/articles/s41598-025-18622-6>
- Expert persona - <https://arxiv.org/abs/2311.10054>
- Least-to-Most prompting - <https://arxiv.org/abs/2205.10625>
- Multi-persona prompting - <https://arxiv.org/abs/2307.05300>
- Prompt repetition - <https://arxiv.org/abs/2512.14982>
- Reflexion - <https://arxiv.org/abs/2303.11366>
- ReWOO (Reasoning Without Observation) - <https://arxiv.org/abs/2305.18323>
- Self-consistency - <https://arxiv.org/abs/2203.11171>
- Similarity-based selection - <https://arxiv.org/abs/2101.06804>
- Skeleton-of-Thought (SoT) - <https://arxiv.org/abs/2307.15337>

52. Step-back prompting - <https://arxiv.org/abs/2310.06117>
54. Tree of Thoughts (ToT) - <https://arxiv.org/abs/2305.10601>

Articles, guides, & named-author resources

56. A/B testing prompts - <https://dev.to/benchwright/how-to-ab-test-llm-prompts-without-breaking-production-4823>
58. Code generation - <https://www.promptingguide.ai/applications/coding>
60. Contrastive examples - https://learnprompting.org/docs/advanced/thought_generation/contrastive_cot
62. Cumulative reasoning - https://learnprompting.org/docs/advanced/self_criticism/cumulative_reasoning
64. Delimiters and sections - <https://www.promptingguide.ai/introduction/settings>
66. Emotional prompting - https://learnprompting.org/docs/advanced/zero_shot/emotion_prompting
68. Few-shot prompting - <https://www.promptingguide.ai/techniques/fewshot>
70. Jailbreak resistance - <https://www.promptfoo.dev/blog/jailbreaking-vs-prompt-injection/>
72. Negative prompting (constraints) - <https://www.the-main-thread.com/p/prompting-like-a-parent>
74. One-shot prompting - https://learnprompting.org/docs/basics/few_shot
76. Prompt chaining - https://www.promptingguide.ai/techniques/prompt_chaining
78. Question answering - <https://www.promptingguide.ai/techniques/rag>
80. Retrieval-Augmented Generation (RAG) - <https://aws.amazon.com/what-is/retrieval-augmented-generation/>
82. Stratified sampling - <https://www.prompthub.us/blog/the-few-shot-prompting-guide>
84. Temperature - https://learnprompting.org/docs/intermediate/configuration_hyperparameters
86. Zero-shot prompting - <https://www.promptingguide.ai/techniques/zeroshot>
53. Thread of Thought (ThoT) - <https://arxiv.org/abs/2311.08734>
55. Universal Self-Consistency - <https://arxiv.org/abs/2311.17311>
57. Analogical prompting - https://learnprompting.org/docs/advanced/thought_generation/analogical_prompting
59. Contextual prompting - <https://handsonai.info/agent-building-blocks/prompts/prompt-engineering/contextual-prompting/>
61. Creative writing - <https://www.promptingguide.ai/techniques>
63. Data extraction - https://www.promptingguide.ai/applications/function_calling
65. Directional stimulus prompting - <https://www.promptingguide.ai/techniques/dsp>
67. Excessive scaffolding for capable models - <https://www.mindstudio.ai/blog/how-to-prompt-gpt-5-5-outcome-first-prompting>
69. Generated knowledge prompting - <https://www.promptingguide.ai/techniques/knowledge>
71. Meta-prompting - <https://www.prompthub.us/blog/a-complete-guide-to-meta-prompting>
73. OCR and document understanding - <https://www.datacamp.com/blog/top-vision-language-models>
75. Program-Aided Language (PAL) - <https://www.promptingguide.ai/techniques/pal>
77. Prompt versioning - <https://www.braintrust.dev/articles/what-is-prompt-versioning>
79. Red-teaming prompts - <https://www.promptfoo.dev/docs/red-team/>
81. Role prompting - <https://learnprompting.org/docs/basics/roles>
83. Summarization - <https://www.prompthub.us/blog/better-summarization-with-chain-of-density-prompting>
85. Zero-shot CoT - <https://www.promptingguide.ai/techniques/cot>